

Algorithm Analysis Documentation

Birthday Cake Candles

Brute Force · Decrease & Conquer · Divide & Conquer

Algorithm 1 — Brute Force (Iterative)

Pseudocode

```
ALGORITHM BirthdayCakeCandles(Candles[n], n) {  
    counter ← 0                                → (1)  
    max ← -1                                    → (1)  
    For i ← 0 to n-1 do                        → (n+1)  
        If max < Candles[i] Then              → (n)  
            max ← Candles[i]                  → (n)  
    For i ← 0 to n-1 do                        → (n+1)  
        If max == Candles[i] Then             → (n)  
            counter ← counter + 1             → (n)  
    Return counter                             → (1)  
}
```

Calculation of T(n)

$T(n) = 1 + 1 + (n+1) + n + n + (n+1) + n + n + 1$

Constants : $1 + 1 + 1 + 1 + 1 = 5$

n-terms : $n + n + n + n + n + n = 6n$

$T(n) = 6n + 5$

Time Complexity

$O(n)$ — Linear

Algorithm 2 — Decrease & Conquer (Recursive)

Pseudocode

```

ALGORITHM BirthdayCakeCandles(candles, n, index, maxH, count) {
  If index == n Then                                     → (1)
    Return count                                         → (1)
  If candles[index] > maxH Then                           → (1)
    maxH ← candles[index]                               → (1)
    count ← 1                                             → (1)
  Else If candles[index] == maxH Then                     → (1)
    count ← count + 1                                    → (1)
  Return BirthdayCakeCandles(candles, n, index+1,        → T(n-1)
                             maxH, count)
}

```

Complexity Analysis — 3 Methods

The recurrence relation for this algorithm is: $T(n) = T(n-1) + c$

Method 1 – Iterative	Method 2 – Master Theorem	Method 3 – Recursion Tree
$T(n) = T(n-1) + c$ $= T(n-2) + 2c$ $= T(n-3) + 3c$ $\dots$ $= T(n-k) + kc$ Set $k = n-1$: $T(n) = T(1) + (n-1)c$ $= c \cdot n$ $\therefore T(n) = O(n)$	$T(n) = aT(n/b) + f(n)$ $a=1, b=1, f(n)=c$ NOT directly applicable because $b=1$ (not > 1). Use substitution or Akra-Bazzi instead. Result from substitution: $\therefore T(n) = O(n)$	Level 0: c Level 1: c Level 2: c $\dots$ Level $n-1$: c Total levels = n Cost/level = c Total = $n \times c$ $\therefore T(n) = O(n)$

Time Complexity	$O(n)$ — Linear
-----------------	-----------------

Algorithm 3 — Divide & Conquer (Recursive)

Pseudocode

```

ALGORITHM BirthdayCakeCandles(candles, low, high, *maxH) {
  If low == high Then                                     → (1)
    maxH ← candles[low]                                   → (1)
    Return 1                                              → (1)
  mid ← (low + high) / 2                                  → (1)
  left_count ← BirthdayCakeCandles(candles, low, mid,    → T(n/2)
                                   left_max)
  right_count ← BirthdayCakeCandles(candles, mid+1, high, → T(n/2)
                                   right_max)
  If left_max > right_max Then                             → (1)
    maxH ← left_max                                       → (1)
}

```

```

    Return left_count          → (1)
Else If right_max > left_max Then → (1)
    maxH ← right_max
    Return right_count          → (1)
Else                            → (1)
    maxH ← left_max             → (1)
    Return left_count + right_count → (1)
}

```

Complexity Analysis — 3 Methods

The recurrence relation for this algorithm is: $T(n) = 2T(n/2) + c$

Method 1 – Iterative	Method 2 – Master Theorem	Method 3 – Recursion Tree
$ \begin{aligned} T(n) &= 2T(n/2) + c \\ &= 2[2T(n/4) + c] + c \\ &= 4T(n/4) + 2c + c \\ &= 4T(n/4) + 3c \\ &\dots \\ &= 2^k \cdot T(n/2^k) + k \cdot c \end{aligned} $ Set $n/2^k = 1 \rightarrow k = \log_2 n$: $ \begin{aligned} T(n) &= n \cdot T(1) + c \cdot \log_2 n \\ &= n + c \cdot \log_2 n \end{aligned} $ $\therefore T(n) = O(n)$	$ \begin{aligned} T(n) &= aT(n/b) + f(n) \\ a=2, \quad b=2, \quad f(n)=c=O(1) \end{aligned} $ $ \begin{aligned} n^{\{\log_b a\}} &= n^{\{\log_2 2\}} \\ &= n^1 = n \end{aligned} $ $ \begin{aligned} f(n) &= O(1) = O(n^0) \\ n^1 &> n^0 \\ &\rightarrow \text{Case 1 of Master Thm.} \end{aligned} $ $\therefore T(n) = \Theta(n) = O(n)$	Level 0: c (1 node) Level 1: c+c (2 nodes) Level 2: c×4 (4 nodes) ... Level k: c×2^k Height = log₂n Leaves = n $ \begin{aligned} \text{Total} &= c \times 2^0 + c \times 2^1 \\ &\quad + \dots + c \times 2^{\log_2 n} \\ &= c(2n-1) \approx 2cn \end{aligned} $ $\therefore T(n) = O(n)$

Time Complexity	$O(n)$ — Linear (Master Theorem – Case 1)
-----------------	---

Algorithms Comparison

All three algorithms share the same asymptotic complexity across all cases:

Best Case = Average Case = Worst Case = $O(n)$

Comparison Criteria	Brute Force (Iterative)	Decrease & Conquer (Recursive)	Divide & Conquer (Recursive)
Time Analysis	$T(n) = 6n + 5$	$T(n) = T(n-1) + c$	$T(n) = 2T(n/2) + c$
Time Complexity	$O(n)$	$O(n)$	$O(n)$
Space Complexity	$O(1)$	$O(n)$	$O(\log n)$
Performance	Fastest	Slowest	Average
Implementation	Loops	Function calls	Function calls

Comparison Criteria	Brute Force (Iterative)	Decrease & Conquer (Recursive)	Divide & Conquer (Recursive)
		& Base case	& Base case
Robustness (Stability)	Stable (Safe)	Unstable (High Risk)	Average (Safe for typical n)
Execution Type	Serial	Serial	Parallel
Stack Usage	None	O(n) stack frames	O(log n) stack frames

Conclusion

Although all three algorithms run in $O(n)$ time, they differ significantly in space usage and implementation complexity:

- Brute Force (Iterative): Best practical choice — $O(1)$ space, no recursion overhead, cache-friendly, stable.
- Decrease & Conquer: Elegant but uses $O(n)$ stack space; risky for very large inputs (stack overflow).
- Divide & Conquer: Uses only $O(\log n)$ stack space — safer than D&C recursion — and naturally parallelizable, though adds code complexity.

EXAMPLES:

Input:

4

3 2 1 3

Output:

2

1: Brute-Force

```

3  #include <stdio.h>
4  int BirthdayCakeCandle(int n, int candles[])
5  {
6      int counter = 0, max = -1;
7      for (int i = 0; i < n; i++)
8      {
9          if (max < candles[i])
10             max = candles[i];
11      }
12      for (int i = 0; i < n; i++)
13      {
14          if (max == candles[i])
15             counter++;
16      }
17      return counter;
18  }
19  int main()
20  {
21      int n;
22      printf("Enter number of candles: ");
23      scanf("%d", &n);
24      int candles[n];
25      for (int i = 0; i < n; i++)
26      {
27          printf("Enter candle no.%d: ", i + 1);
28          scanf("%d", &candles[i]);
29      }
30
31      printf("%d", BirthdayCakeCandle(n, candles));
32  }
33

```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

● PS C:\Users\MMS\OneDrive\Desktop\AlgoTask> & 'c:\Users\MMS\.vscode\extensions\ms-vs-
n\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-ng2hyihl.ark' '--stdout=
icrosoft-MIEngine-Error-xjp2sx5s.2fb' '--pid=Microsoft-MIEngine-Pid-yrqxx1n2.pnu' '
reter=mi'
Enter number of candles: 4
Enter candle no.1: 3
Enter candle no.2: 2
Enter candle no.3: 1
Enter candle no.4: 3
2

```

2: Decrease and Conquer

```

3  #include <stdio.h>
4  int BirthdayCakeCandle(int candles[], int n, int index, int *maxH, int count)
5  {
6      if (index == n)
7      {
8          return count;
9      }
10     if (candles[index] > *maxH)
11     {
12         *maxH = candles[index];
13         count = 1;
14     }
15     else if (candles[index] == *maxH)
16     {
17         count++;
18     }
19     return BirthdayCakeCandle(candles, n, index + 1, maxH, count);
20 }
21 int main()
22 {
23     int n;
24     printf("Enter number of candles: ");
25     scanf("%d", &n);
26     int candles[n];
27     for (int i = 0; i < n; i++)
28     {
29         printf("Enter candle no.%d: ", i + 1);
30         scanf("%d", &candles[i]);
31     }
32     int maxH = 0;
33     printf("%d", BirthdayCakeCandle(candles, n, 0, &maxH, 0));
34 }

```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

● PS C:\Users\MMS\OneDrive\Desktop\AlgoTask> & 'c:\Users\MMS\.vscode\extensions\ms-vscode.c\n\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-ntnx5rnm.sai' '--stdout=Microsoft-MIEngine-Error-z1kii54h.rnj' '--pid=Microsoft-MIEngine-Pid-zignpwm3.mi5' '--dbgExreter=mi'

Enter number of candles: 4

Enter candle no.1: 3

Enter candle no.2: 2

Enter candle no.3: 1

Enter candle no.4: 3

2

3: Divide & Conquer

```

4  int BirthdayCakeCandles(int candles[], int low, int high, int *maxH)
5  {
6      if (low == high)
7      {
8          *maxH = candles[low];
9          return 1;
10     }
11     int mid = (low + high) / 2;
12     int left_max, right_max;
13     int left_count = BirthdayCakeCandles(candles, low, mid, &left_max);
14     int right_count = BirthdayCakeCandles(candles, mid + 1, high, &right_max);
15     if (left_max > right_max)
16     {
17         *maxH = left_max;
18         return left_count;
19     }
20     else if (right_max > left_max)
21     {
22         *maxH = right_max;
23         return right_count;
24     }
25     else
26     {
27         *maxH = left_max;
28         return left_count + right_count;
29     }
30 }
31 int main()
32 {
33     int n;
34     printf("Enter number of candles: ");
35     scanf("%d", &n);
36     int candles[n];
37     for (int i = 0; i < n; i++)
38     {
39         printf("Enter candle no.%d: ", i + 1);
40         scanf("%d", &candles[i]);
41     }
42     int maxH = 0;
43     printf("%d", BirthdayCakeCandles(candles, 0, n - 1, &maxH));
44 }
45

```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\WMS\OneDrive\Desktop\AlgoTask> & 'c:\Users\WMS\.vscode\extensions\ms-vscode.cpptools-1.32.2-cx.s4i' '--stdout=Microsoft-MIEngine-Out-xamfk454.2gd' '--stderr=Microsoft-MIEngine-Error-wjuyg4we.w3x' 'preter=mi'
Enter number of candles: 4
Enter candle no.1: 3
Enter candle no.2: 2
Enter candle no.3: 1
Enter candle no.4: 3
2

```